

Практична робота №5

Взаємодія фронтенду та бекенду: AJAX, обмін даними JSON

ІО-43 Череватов Артем

Варіант 20

Крок 1. Підготувати JSON-ендпоінт списку

Створено файл `api_list.php`, у якому підключено `db.php`, встановлено заголовок `header('Content-Type: application/json')`, виконано вибірку даних із таблиці `teams` та додано обробку параметра фільтрації `$_GET['q']` для пошуку за містом.

```
api_list.php
1  <?php
2  require_once 'db.php';
3  // Встановлюємо заголовок для повернення даних у форматі JSON
4  header('Content-Type: application/json');
5
6  $cityQuery = $_GET['q'] ?? '';
7
8  try {
9      if ($cityQuery !== '') {
10         // Пошук: фільтрація за містом
11         $stmt = $pdo->prepare('SELECT * FROM teams WHERE city LIKE :city ORDER BY points DESC');
12         $stmt->execute([':city' => '%' . $cityQuery . '%']);
13         $rows = $stmt->fetchAll();
14     } else {
15         // Виведення всієї турнірної таблиці, відсортованої за очками
16         $rows = $pdo->query('SELECT * FROM teams ORDER BY points DESC')->fetchAll();
17     }
18     echo json_encode($rows);
19 } catch (PDOException $e) {
20     http_response_code(500);
21     echo json_encode(['error' => $e->getMessage()]);
22 }
23 ?>
```

Крок 2. Підготувати сторінку з контейнером для результатів

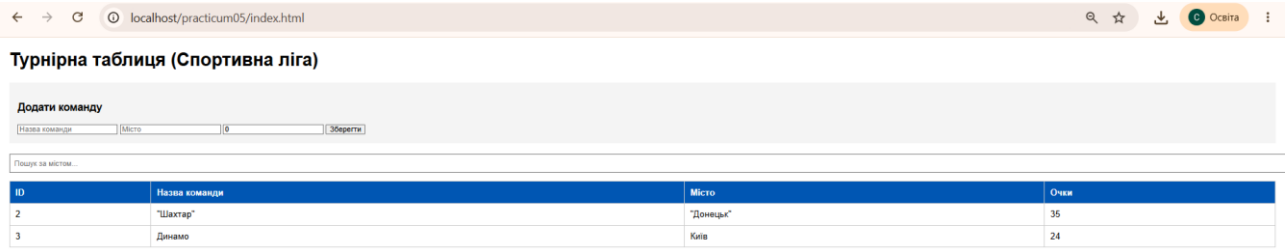
Створено головну сторінку, яка містить поле пошуку (`<input id="search">`), форму додавання запису, блок для виведення помилок та порожній контейнер (`<tbody id="results">`) для динамічного виведення турнірної таблиці без використання PHP-виводу таблиці всередині HTML.

```
<input type="text" id="search" placeholder="Пошук за містом..." style="width: 100%; padding: 8px;">

<table>
  <thead>
    <tr>
      <th>ID</th>
      <th>Назва команди</th>
      <th>Місто</th>
      <th>Очки</th>
    </tr>
  </thead>
  <!-- Порожній контейнер для результатів [cite: 12] -->
  <tbody id="results"></tbody>
</table>
```

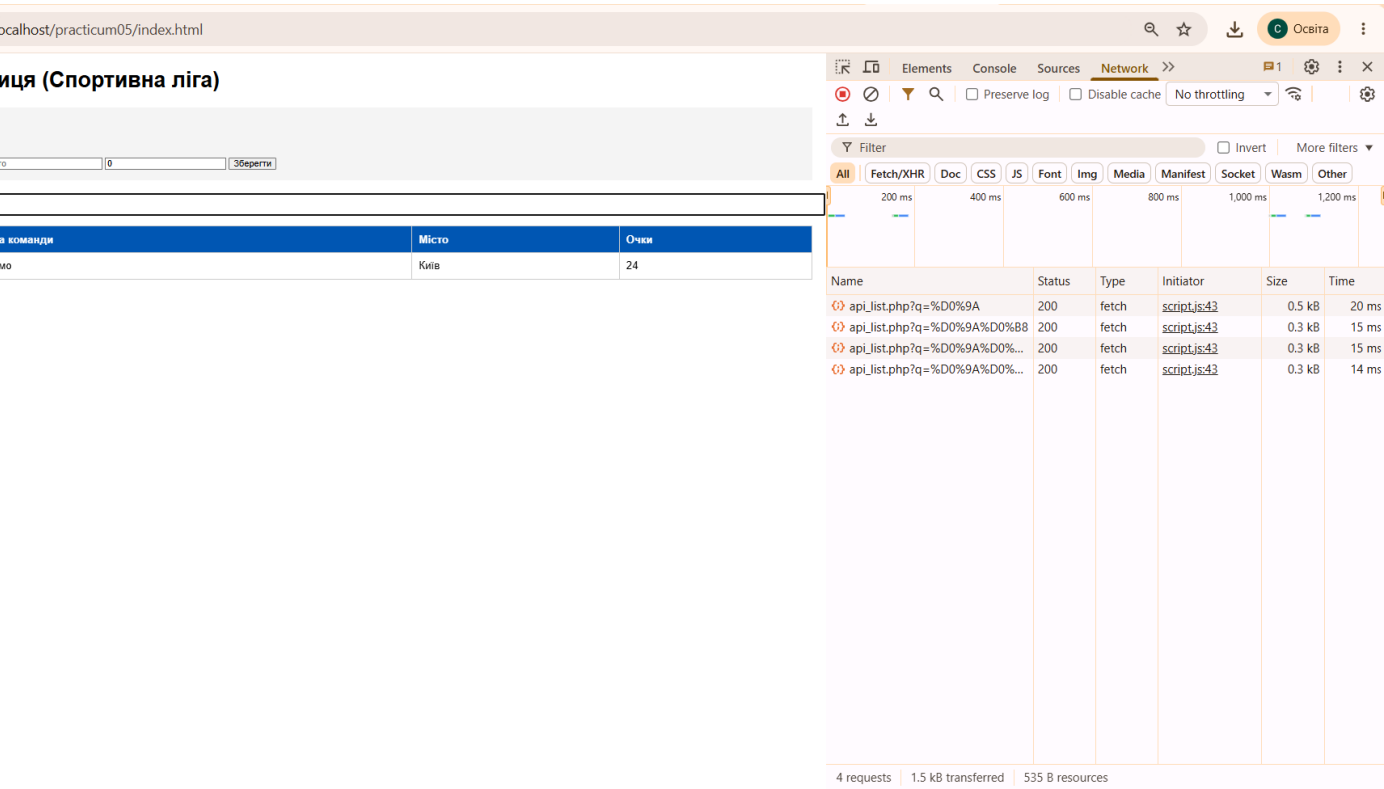
Крок 3. Завантажити список через AJAX при відкритті сторінки

У файлі script.js реалізовано функцію завантаження даних за допомогою сучасного метода `fetch('api_list.php')`, яка перетворює відповідь методом `.json()` та виводить отримані дані в DOM-контейнер одразу після завантаження сторінки.



Крок 4. Реалізувати живий пошук без перезавантаження

На подію `input` поля пошуку налаштовано відправку асинхронного запиту `fetch('api_list.php?q=' + encodeURIComponent(значення))`, після чого отримується відфільтрований список і відбувається перемалювання контейнера на льоту.



Крок 5. Підготувати ендпоінт додавання запису

Створено файл `api_add.php`, який приймає дані з POST-запиту, виконує захищений INSERT-запит через підготовлені вирази та повертає JSON із доданим записом та статус-кодом 201.

```
api_add.php
1 <?php
2 require_once 'db.php';
3 header('Content-Type: application/json');
4
5 if ($_SERVER['REQUEST_METHOD'] === 'POST') {
6     $name = $_POST['name'] ?? '';
7     $city = $_POST['city'] ?? '';
8     $points = $_POST['points'] ?? 0;
9
10    if (!empty($name) && !empty($city)) {
11        $stmt = $pdo->prepare('INSERT INTO teams (name, city, points) VALUES (:name, :city, :points)');
12        $stmt->execute([
13            ':name' => $name,
14            ':city' => $city,
15            ':points' => $points
16        ]);
17
18        // Повертаємо код 201 та JSON про успіх [cite: 12]
19        http_response_code(201);
20        echo json_encode(['status' => 'success', 'id' => $pdo->lastInsertId()]);
21    } else {
22        http_response_code(400);
23        echo json_encode(['error' => 'Некоректні дані']);
24    }
25 }
26
```

Крок 6. Додати запис через AJAX без перезавантаження сторінки

Оброблено відправлення форми через подію `event.preventDefault()`, викликано запит `fetch('api_add.php', {method: 'POST', ...})`, після чого список автоматично оновлюється новим рядком у контейнері без перезавантаження всієї сторінки.

← → ↺ localhost/practicum05/index.html

Турнірна таблиця (Спортивна ліга)

Додати команду

Назва команди Місто

Пошук за містом...

ID	Назва команди	Місто	Очки
2	"Шахтар"	"Донецьк"	35
4	Полісся	Житомир	28
3	Динамо	Київ	24

Elements Console Sources Network >> 1 Освіта

Filter

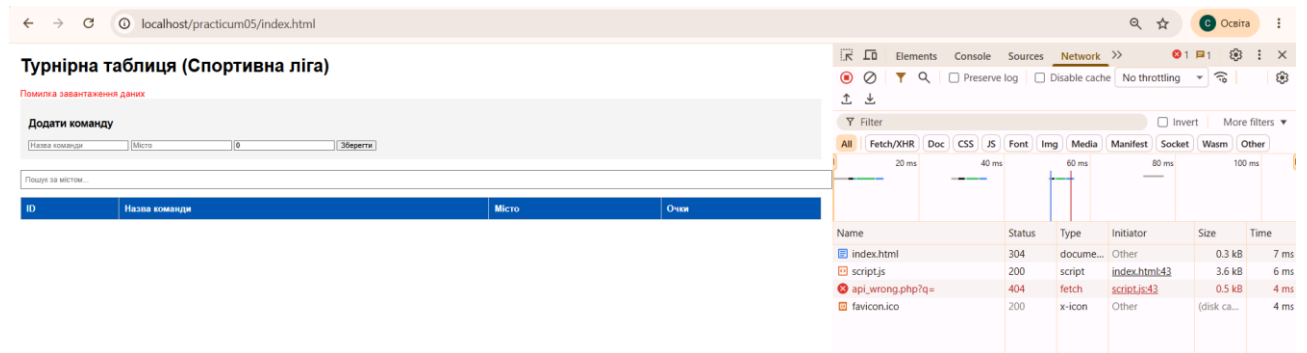
All Fetch/XHR Doc CSS JS Font Img Media Manifest Socket Wasm Other

Name	Status	Type	Initiator	Size	Time
api_add.php	201	fetch	script.js:22	0.3 kB	15 ms
api_list.php?q=	200	fetch	script.js:43	0.6 kB	7 ms

2 requests 880 B transferred 383 B resources

Крок 7. Обробити помилки мережі та сервера

У клієнтському скрипті налаштовано перевірку `response.ok` та використання блока `catch()` для відловлювання мережевих помилок. При виникненні проблем повідомлення виводиться безпосередньо в спеціальний контейнер на сторінці.



Крок 8. Перевірити результат в інструментах розробника

За допомогою відкриття вкладки **Network** в інструментах розробника перевірено, що запити до `api_list.php` та `api_add.php` мають тип `fetch`, а вкладка **Elements** підтверджує оновлення DOM без повторного завантаження документа.

api_list.php?q=R	200	fetch	script.js:43	0.2 kB	16 ms
api_list.php?q=Rb	200	fetch	script.js:43	0.2 kB	12 ms
api_list.php?q=R	200	fetch	script.js:43	0.2 kB	7 ms
api_list.php?q=	200	fetch	script.js:43	0.6 kB	14 ms
api_list.php?q=%D0%9A	200	fetch	script.js:43	0.5 kB	15 ms
api_list.php?q=%D0%9A%D0%B8	200	fetch	script.js:43	0.3 kB	15 ms
api_list.php?q=%D0%9A%D0%...	200	fetch	script.js:43	0.3 kB	16 ms
api_list.php?q=%D0%9A%D0%...	200	fetch	script.js:43	0.3 kB	17 ms
api_add.php	201	fetch	script.js:22	0.3 kB	15 ms